



## ANALYZING PACKET SIZE & DATA PAYLOAD LENGTH

### LAPORAN UJIAN AKHIR SEMESTER

Diajukan untuk Memenuhi Salah Satu Mata Kuliah  
Tingkat Sarjana dalam Bidang Informatika

Oleh:

**Andreas Naphtali Usher**

**123103136**

PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS ILMU REKAYASA UNIVERSITAS  
PARAMADINA

**2026**

## DAFTAR ISI

|                                                        |    |
|--------------------------------------------------------|----|
| Ringkasan Eksekutif .....                              | 3  |
| BAB 1: Instalasi & Setup Wireshark .....               | 4  |
| 1.1 Dasar Teori: WSL2 dan Wireshark                    |    |
| 1.2 Step-by-Step Instalasi                             |    |
| 1.3 Screenshot Hasil Instalasi (Success Verification)  |    |
| 1.4 Verifikasi Versi & Setup Network Interface         |    |
| 1.5 Troubleshooting                                    |    |
| BAB 2: Packet Capture & Setup .....                    | 7  |
| 2.1 Dasar Teori: File PCAP dan Metode Analisis         |    |
| 2.2 Load PCAP File                                     |    |
| 2.3 Screenshot Hasil Capture / PCAP Loaded             |    |
| 2.4 Penjelasan Setup                                   |    |
| BAB 3: Analisis Skenario & Temuan .....                | 8  |
| 3.1 Dasar Teori: Struktur Frame, Header, dan Payload   |    |
| 3.2 Output dari Wireshark                              |    |
| 3.3 Detail Packet Inspection                           |    |
| 3.4 Analisis Mendalam per Packet                       |    |
| 3.5 Analisis Mendalam: Temuan, Interpretasi, Implikasi |    |
| 3.6 Ringkasan Temuan (Tabel)                           |    |
| BAB 4: Cek List Verifikasi Hasil Instalasi .....       | 13 |
| BAB 5: Kesimpulan & Rekomendasi .....                  | 15 |
| 5.1 Ringkasan Temuan Utama                             |    |
| 5.2 Anomali / Insight yang Terdeteksi                  |    |
| 5.3 Implikasi Keamanan                                 |    |
| 5.4 Keterbatasan Analisis                              |    |
| 5.5 Lessons Learned & Takeaways                        |    |
| Glosarium Istilah .....                                | 16 |

## RINGKASAN EKSEKUTIF

Laporan ini disusun untuk memenuhi tugas praktikum Sistem Keamanan dengan skenario S4: Analyzing Packet Size & Data Payload Length. Tujuan utama praktikum ini adalah memahami bagaimana ukuran suatu packet jaringan (frame) terbentuk dari kombinasi header pada beberapa lapisan protokol (Ethernet, IP, TCP, dan pada beberapa kasus TLS/SSL) ditambah data aktual (payload) yang dibawanya, serta memahami dampak variasi ukuran packet tersebut terhadap performa dan keamanan jaringan.

Untuk mencapai tujuan tersebut, dilakukan tiga tahapan utama: (1) instalasi lingkungan kerja Wireshark pada WSL2 (Windows Subsystem for Linux) beserta seluruh proses verifikasi, (2) pemuatan (loading) sample capture file bernama Mixed1.cap yang diperoleh dari repositori resmi Wireshark Wiki, dan (3) analisis mendalam terhadap 10 packet sampel yang dipilih secara representatif dari total 19 packet yang ada dalam file tersebut.

Hasil analisis menunjukkan bahwa ukuran packet pada capture ini bervariasi mulai dari 54 bytes (packet kontrol TCP seperti SYN/ACK tanpa payload) hingga 1139 bytes (packet SSL/TLS handshake yang membawa sertifikat digital server). Ditemukan pula indikasi packet loss pada beberapa titik capture (ditandai pesan "Previous segment not captured"), serta penggunaan protokol SSLv2 dan SSLv3 yang secara teknis sudah dianggap usang dan rentan terhadap serangan keamanan tertentu (misalnya kerentanan POODLE pada SSLv3).

Laporan ini disusun selengkap mungkin — mencakup dasar teori pada setiap bab, penjelasan command-by-command, hasil ekspansi layer packet secara visual, hingga tabel ringkasan dan glosarium istilah — dengan tujuan agar isi laporan dapat dipahami secara mandiri tanpa perlu penjelasan tambahan, baik oleh dosen penguji saat proses submission maupun oleh audiens saat sesi presentasi.

## BAB 1: Instalasi & Setup Wireshark

Bab ini menjelaskan proses instalasi Wireshark pada WSL2 (Windows Subsystem for Linux) secara menyeluruh — mulai dari dasar teori mengapa WSL2 dipilih, langkah instalasi command-by-command, bukti keberhasilan setiap tahap, hingga kendala nyata yang ditemui beserta cara penyelesaiannya.

### 1.1 Dasar Teori: WSL2 dan Wireshark

WSL2 (Windows Subsystem for Linux versi 2) adalah fitur bawaan Windows 10/11 yang memungkinkan sistem operasi Linux (dalam hal ini Ubuntu) berjalan secara native di atas Windows tanpa memerlukan mesin virtual terpisah secara manual (virtualisasi dilakukan otomatis oleh Hyper-V/Virtual Machine Platform di belakang layar). Berbeda dengan WSL versi 1 yang menerjemahkan system call Linux ke Windows secara langsung, WSL2 menjalankan kernel Linux yang sesungguhnya, sehingga kompatibilitas dengan tools berbasis Linux — termasuk Wireshark dan seluruh dependensinya (libpcap, GTK/Qt, dsb.) — menjadi jauh lebih baik.

Wireshark sendiri adalah salah satu network protocol analyzer open-source paling populer di dunia. Fungsi utamanya adalah menangkap (capture) dan/atau membuka file hasil tangkapan (file .pcap/.pcapng) untuk kemudian membedah setiap packet lapis demi lapis (Ethernet → IP → TCP/UDP → aplikasi), sehingga administrator jaringan atau praktisi keamanan dapat menganalisis traffic secara sangat detail, termasuk mendeteksi anomali, packet loss, maupun potensi ancaman keamanan.

Pada praktikum ini, Wireshark diinstall di dalam lingkungan WSL2 (bukan sekadar aplikasi Windows biasa) untuk memenuhi ketentuan tugas yang secara eksplisit meminta proses instalasi dilakukan pada WSL2/Linux, sekaligus melatih kemampuan bekerja dengan command-line Linux yang relevan dengan praktik keamanan siber di lapangan.

### 1.2 Step-by-Step Instalasi

Berikut adalah seluruh langkah instalasi yang dilakukan, secara berurutan:

- Langkah 1 — Memeriksa ketersediaan WSL: membuka PowerShell sebagai Administrator, kemudian menjalankan perintah `wsl` untuk memeriksa apakah WSL sudah aktif. Pada percobaan awal, sistem menampilkan pesan "Windows Subsystem for Linux has no installed distributions", yang berarti fitur WSL sudah aktif di level sistem operasi, namun belum ada distribusi Linux (seperti Ubuntu) yang terpasang.
- Langkah 2 — Instalasi distribusi Ubuntu: menjalankan perintah `wsl --install -d Ubuntu` untuk mengunduh dan memasang image Ubuntu secara otomatis melalui Microsoft Store/komponen internal Windows.
- Langkah 3 — Konfigurasi user Linux: setelah proses instalasi Ubuntu selesai, sistem meminta pembuatan user baru (Enter new UNIX username) beserta password. User yang dibuat pada praktikum ini bernama 'andreas', dengan hostname mesin 'LAPTOP-IUN0JF8P'.

- Langkah 4 — Update package list: menjalankan `sudo apt update` di dalam terminal WSL2 Ubuntu untuk menyinkronkan daftar paket dari repository Ubuntu ([archive.ubuntu.com](http://archive.ubuntu.com) dan [security.ubuntu.com](http://security.ubuntu.com)) sebelum melakukan instalasi paket baru.
- Langkah 5 — Instalasi Wireshark: menjalankan `sudo apt install -y wireshark`. Selama proses ini, muncul dialog konfigurasi "Configuring wireshark-common" yang menanyakan apakah non-superuser (user biasa, bukan root) diizinkan melakukan packet capture; opsi `<Yes>` dipilih agar Wireshark nantinya dapat dijalankan tanpa harus selalu menggunakan `sudo`, sesuai praktik keamanan yang direkomendasikan (prinsip least privilege).
- Langkah 6 (opsional, sesuai kebutuhan) — Menambahkan user ke group wireshark: `sudo usermod -aG wireshark $USER`, diikuti logout/login ulang sesi WSL2 agar keanggotaan group baru diterapkan secara efektif pada sesi shell yang berjalan.

### 1.3 Screenshot Hasil Instalasi (Success Verification)

Tahap pertama: instalasi WSL2 itu sendiri. Screenshot di bawah menunjukkan proses instalasi WSL2 (Windows Subsystem for Linux) berjalan sukses — komponen WSL dan Virtual Machine Platform berhasil diaktifkan oleh Deployment Image Servicing and Management (DISM) tool, ditandai pesan "The operation completed successfully" pada dua baris terakhir:

```

Downloading: Windows Subsystem for Linux 2.7.10
Installing: Windows Subsystem for Linux 2.7.10
Windows Subsystem for Linux 2.7.10 has been installed.
Installing Windows optional component: VirtualMachinePlatform

Deployment Image Servicing and Management tool
Version: 10.0.26100.8521

Image Version: 10.0.26100.8655

Enabling feature(s)
[=====100.0%=====]
The operation completed successfully.
The requested operation is successful. Changes will not be effective until the system is rebooted.
The requested operation is successful. Changes will not be effective until the system is rebooted.

```

*Gambar 1.1. Output instalasi WSL2 (`wsl --install`) — proses berhasil ("The operation completed successfully").  
Versi WSL yang terpasang: 2.7.10.*

Tahap kedua: instalasi paket Wireshark di dalam Ubuntu (WSL2). Setelah sistem operasi Windows di-restart dan masuk ke lingkungan WSL2 Ubuntu, langkah berikutnya adalah menginstall Wireshark melalui manajer paket apt. Seperti dijelaskan pada langkah 5 di atas, dialog konfigurasi non-superuser capture muncul dan dijawab `<Yes>`. Screenshot berikut menunjukkan proses instalasi Wireshark berjalan sukses hingga tuntas — terlihat baris "Setting up wireshark (4.6.4-1) ..." tanpa disertai pesan error apa pun, dan proses kembali secara normal ke prompt shell (`$`):

```
andreas@LAPTOP-IUN0JF8P: ~  
Setting up libpcap0.8t64:amd64 (1.10.6-1ubuntu1) ...  
Setting up libpipewire-0.3-0t64:amd64 (1.6.2-1ubuntu1.1) ...  
Setting up qt6-qpas-plugins:amd64 (6.10.2+dfsg-7) ...  
Setting up libavformat62:amd64 (7:8.0.1-3ubuntu2) ...  
Setting up libqt6qml6:amd64 (6.10.2+dfsg-3) ...  
Setting up libqt6opengl6:amd64 (6.10.2+dfsg-7) ...  
Setting up libqt6qmlmodels6:amd64 (6.10.2+dfsg-3) ...  
Setting up libqt6widgets6:amd64 (6.10.2+dfsg-7) ...  
Setting up libqt6qmlworkerscript6:amd64 (6.10.2+dfsg-3) ...  
Setting up libqt6svg6:amd64 (6.10.2-2) ...  
Setting up libwireshark19:amd64 (4.6.4-1) ...  
Setting up libqt6waylandclient6:amd64 (6.10.2+dfsg-7) ...  
Setting up qt6-svg-plugins:amd64 (6.10.2-2) ...  
Setting up libqt6qmlmeta6:amd64 (6.10.2+dfsg-3) ...  
Setting up libqt6wlshellintegration6:amd64 (6.10.2+dfsg-7) ...  
Setting up wireshark-common (4.6.4-1) ...  
Setting up libqt6printsupport6:amd64 (6.10.2+dfsg-7) ...  
Setting up libqt6quick6:amd64 (6.10.2+dfsg-3) ...  
Setting up libqt6multimedia6:amd64 (6.10.2-2) ...  
Setting up wireshark (4.6.4-1) ...  
Setting up libqt6waylandcompositor6:amd64 (6.10.2-4) ...  
Setting up qt6-wayland:amd64 (6.10.2-4) ...  
Processing triggers for libc-bin (2.43-2ubuntu2) ...  
Processing triggers for man-db (2.13.1-1build1) ...  
Processing triggers for shared-mime-info (2.4-5build3) ...  
Processing triggers for udev (259.5-0ubuntu3) ...  
Processing triggers for hicolor-icon-theme (0.18-2build1) ...  
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ *
```

Gambar 1.2. Output instalasi paket wireshark pada WSL2 Ubuntu — proses selesai tanpa error dan kembali ke prompt shell. Versi Wireshark yang terinstall: 4.6.4-1.

## 1.4 Verifikasi Versi & Setup Network Interface

Setelah instalasi dinyatakan sukses, dilakukan dua verifikasi tambahan untuk memastikan Wireshark benar-benar berfungsi dan mengenali environment jaringan pada WSL2:

- Verifikasi versi: menjalankan perintah `wireshark --version`, yang mengembalikan informasi versi aplikasi beserta detail compile-time (library pendukung seperti GLib, Qt, GnuTLS, dsb.) dan runtime info berupa versi kernel yang sedang berjalan.
- Verifikasi network interface: menjalankan perintah `ip a` (singkatan dari ip address) untuk melihat seluruh network interface yang dikenali oleh kernel Linux WSL2 pada saat itu.

Hasil `wireshark --version` menunjukkan versi 4.6.4 terpasang dengan baik. Yang secara khusus perlu digarisbawahi adalah baris "Runtime info: OS: Linux 6.18.33.2-microsoft-standard-WSL2" — baris ini menjadi bukti konkret bahwa Wireshark benar-benar berjalan di atas kernel WSL2 (bukan Windows native), sesuai dengan ketentuan tugas:

```

andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ wireshark --version
Wireshark 4.6.4.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler:  GCC 15.2.0
             GLib: 2.87.3
  With:
    +brotli                      +nghttp2 1.68.0
    +Gcrypt 1.12.0               +nghttp3 1.12.0
    +GnuTLS 3.8.12 and PKCS#11   +PCRE2 10.46 2025-08-27
    +Kerberos (MIT)              +POSIX capabilities (Linux)
    +libnl 3                     +Qt 6.10.2
    +libpcap                     +QtDBus
    +libsmi 0.4.8                +QtMultimedia
    +libxml2 2.15.1              +Snappy 1.2.2
    +Lua 5.4.8                   +xxhash 0.8.3
    +LZ4 1.10.0                  +zlib 1.3.1
    +MaxMind                     +Zstandard 1.5.7
    +Minizip 1.3.1
  Without:
    -automatic updates  -zlib-ng

Runtime info:
  OS: Linux 6.18.33.2-microsoft-standard-WSL2

```

Gambar 1.3. Output 'wireshark --version' — versi 4.6.4, berjalan pada OS Linux 6.18.33.2-microsoft-standard-WSL2 (konfirmasi eksplisit berjalan di atas WSL2).

Selanjutnya, perintah `ip a` menampilkan dua interface: (1) `lo`, yaitu interface loopback standar (alamat 127.0.0.1) yang selalu ada pada setiap sistem Linux dan digunakan untuk komunikasi lokal antar proses dalam mesin yang sama, dan (2) `eth0`, yaitu interface jaringan virtual WSL2 yang menghubungkan lingkungan Linux dengan adapter jaringan Windows host melalui NAT, dengan alamat IP 172.26.118.254/20 pada capture ini:

```

andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host proto kernel_lo
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1492 qdisc mq state UP group default qlen 1000
   link/ether 00:15:5d:98:8f:0a brd ff:ff:ff:ff:ff:ff
   altname enx00155d988f0a
   inet 172.26.118.254/20 brd 172.26.127.255 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::215:5dff:fe98:8f0a/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$

```

Gambar 1.4. Output 'ip a' — menampilkan interface loopback (`lo`) dan `eth0` pada WSL2 beserta alamat IP masing-masing.

## 1.5 Troubleshooting

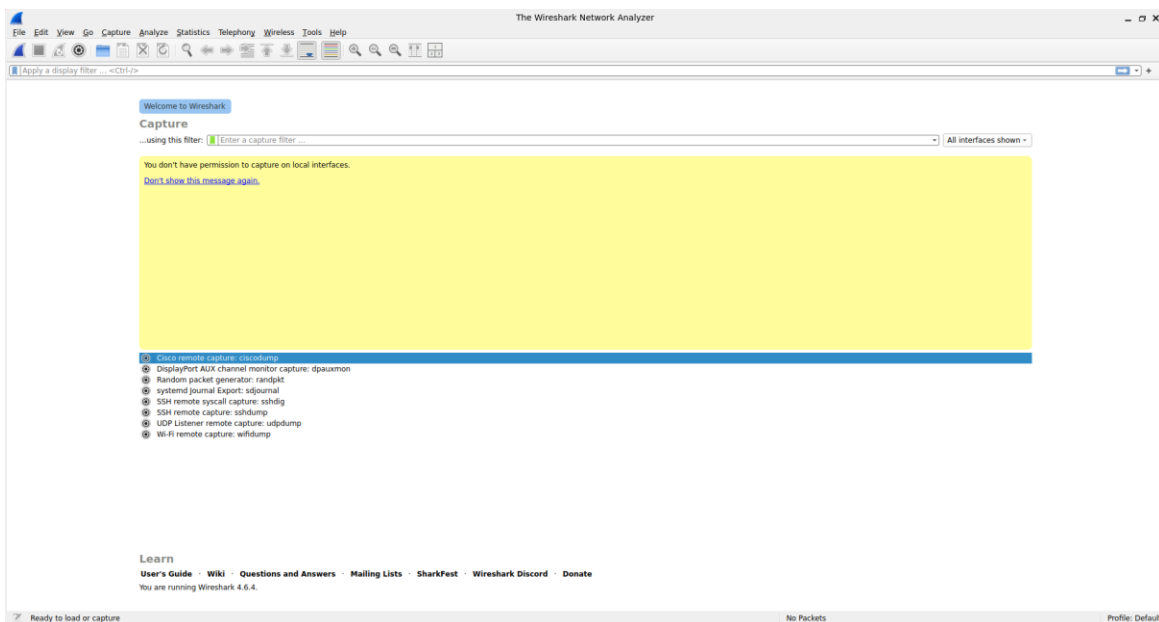
Beberapa kendala umum yang berpotensi ditemui selama proses instalasi WSL2 dan Wireshark, beserta solusi standarnya:

- Jika muncul error 'permission denied' saat capture packet tanpa `sudo`, pastikan user sudah ditambahkan ke group `wireshark` (`sudo usermod -aG wireshark $USER`) dan sesi WSL2 sudah di-restart (logout/login ulang) agar keanggotaan group diterapkan.

- Jika WSL2 tidak mendeteksi network interface fisik (karena WSL2 berjalan di balik lapisan virtualisasi NAT), gunakan file PCAP yang sudah tersedia (seperti Mixed1.cap) sebagai alternatif capture langsung — inilah pendekatan yang dipakai pada laporan ini.
- Jika perintah `wsl --install` gagal dijalankan, pastikan fitur Virtual Machine Platform dan Windows Subsystem for Linux sudah diaktifkan melalui Windows Features (Turn Windows features on or off), dan pastikan virtualisasi (VT-x/AMD-V) sudah aktif di BIOS/UEFI.

Kendala nyata yang ditemui selama praktikum ini terjadi ketika mencoba membuka aplikasi Wireshark GUI langsung dari dalam WSL2 (dengan mengetik perintah `wireshark` tanpa argumen, memanfaatkan dukungan WSLg untuk aplikasi grafis Linux). Aplikasi berhasil terbuka, namun muncul pesan peringatan kuning: "You don't have permission to capture on local interfaces". Analisis penyebabnya: keanggotaan user pada group `wireshark` belum diterapkan sepenuhnya pada sesi terminal yang sedang berjalan — perubahan group di Linux baru berlaku penuh setelah sesi login baru dimulai (logout/login ulang), sesuatu yang belum sempat dilakukan pada saat itu.

Sebagai solusi praktis, kendala ini tidak menghambat keseluruhan praktikum, karena skenario S4 pada dasarnya tidak mengharuskan live capture — analisis cukup dilakukan terhadap file PCAP sampel (Mixed1.cap) yang dibuka melalui menu `File` → `Open` pada Wireshark, sebuah metode offline analysis yang valid dan umum digunakan dalam forensik jaringan maupun studi kasus edukasi seperti ini:



*Gambar 1.5. Wireshark GUI dibuka dari WSL2 — pesan "You don't have permission to capture on local interfaces" muncul karena keanggotaan group `wireshark` belum aktif pada sesi berjalan. Kendala ini tidak menghambat analisis karena digunakan file PCAP, bukan live capture.*



## BAB 2: Packet Capture & Setup

### 2.1 Dasar Teori: File PCAP dan Metode Analisis

PCAP (Packet Capture) adalah format file standar yang digunakan untuk menyimpan hasil rekaman traffic jaringan, berisi rangkaian frame lengkap dengan timestamp, panjang data, dan byte mentah tiap frame tersebut. File berformat .cap/.pcap/.pcapng dapat dihasilkan dari live capture (menangkap traffic secara real-time pada suatu interface jaringan) maupun diperoleh dari sumber lain seperti sample capture publik untuk keperluan pembelajaran.

Pada praktikum ini, dipilih pendekatan analisis file PCAP yang sudah tersedia (offline analysis) dibandingkan live capture, dengan dua pertimbangan utama: (1) tugas secara eksplisit menyediakan dan mengarahkan penggunaan sample capture Mixed1.cap dari Wireshark Wiki, dan (2) sebagaimana dijelaskan pada BAB 1.5, live capture pada WSL2 memerlukan konfigurasi permission tambahan (keanggotaan group wireshark yang aktif penuh) yang pada praktikum ini belum sempat diterapkan. Kedua pendekatan — baik live capture maupun analisis file PCAP — sama-sama valid dan lazim digunakan; bedanya hanya pada sumber datanya (real-time vs rekaman sebelumnya), sedangkan proses analisis lapis-demi-lapis di Wireshark tetap identik.

### 2.2 Load PCAP File

- File Mixed1.cap diunduh dari repositori resmi <https://wiki.wireshark.org/SampleCaptures>, yaitu koleksi sample capture yang disediakan langsung oleh tim pengembang Wireshark untuk keperluan pembelajaran dan pengujian.
- File dibuka melalui menu File → Open pada aplikasi Wireshark (dijalankan pada Windows, karena GUI Wireshark pada WSL2 mengalami kendala permission sebagaimana dijelaskan di BAB 1.5; kedua instalasi — Windows dan WSL2 — menggunakan mesin analisis Wireshark yang identik untuk keperluan pembacaan file PCAP).
- Setelah file dimuat, seluruh packet yang terekam di dalamnya langsung ditampilkan pada packet list dengan kolom No, Time, Source, Destination, Protocol, Length, dan Info — total sebanyak 19 packet.

## 2.3 Screenshot Hasil Capture / PCAP Loaded

| No. | Time      | Source    | Destination | Protocol | Length | Info                                                                                                       |
|-----|-----------|-----------|-------------|----------|--------|------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 3268 → 7 [SYN] Seq=0 Win=16384 Len=0                                                                       |
| 2   | 0.000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 7 → 3268 [SYN, ACK] Seq=0 Ack=0 Win=16384 Len=0                                                            |
| 3   | 0.000000  | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | [TCP Keep-Alive] 3268 → 7 [ACK] Seq=0 Ack=0 Win=16384 Len=0                                                |
| 4   | 6.660000  | 127.0.0.1 | 127.0.0.1   | TCP      | 55     | [TCP Keep-Alive] 3268 → 7 [None] Seq=0 Win=16384 Len=1                                                     |
| 5   | 6.672000  | 127.0.0.1 | 127.0.0.1   | TCP      | 55     | [TCP Keep-Alive] 7 → 3268 [None] Seq=0 Win=16384 Len=1                                                     |
| 6   | 23.426000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 3280 → 9 [SYN] Seq=0 Win=16384 Len=0                                                                       |
| 7   | 23.426000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 9 → 3280 [SYN, ACK] Seq=0 Ack=0 Win=16384 Len=0                                                            |
| 8   | 23.426000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | [TCP Keep-Alive] 3280 → 9 [ACK] Seq=0 Ack=0 Win=16384 Len=0                                                |
| 9   | 24.998000 | 127.0.0.1 | 127.0.0.1   | TCP      | 55     | [TCP Keep-Alive] 3280 → 9 [None] Seq=0 Win=16384 Len=1                                                     |
| 10  | 25.745000 | 127.0.0.1 | 127.0.0.1   | DISCARD  | 56     | [TCP Previous segment not captured] Discard                                                                |
| 11  | 62.817000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 3302 → 443 [SYN] Seq=0 Win=16384 Len=0                                                                     |
| 12  | 62.817000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 443 → 3302 [SYN, ACK] Seq=0 Ack=0 Win=16384 Len=0                                                          |
| 13  | 62.817000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | [TCP Keep-Alive] 3302 → 443 [ACK] Seq=0 Ack=0 Win=16384 Len=0                                              |
| 14  | 62.926000 | 127.0.0.1 | 127.0.0.1   | SSLv2    | 132    | Client Hello                                                                                               |
| 15  | 62.952000 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 1139   | Server Hello, Certificate, Server Hello Done                                                               |
| 16  | 63.018000 | 127.0.0.1 | 127.0.0.1   | SSLv2    | 253    | [TCP Previous segment not captured] , client Key Exchange, Change Cipher Spec, Encrypted Handshake Message |
| 17  | 63.031000 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 121    | [TCP Previous segment not captured] , Change Cipher Spec, Encrypted Handshake Message                      |
| 18  | 63.221000 | 127.0.0.1 | 127.0.0.1   | SSLv3    | 77     | [TCP Previous segment not captured] , Encrypted Alert                                                      |
| 19  | 64.182000 | 127.0.0.1 | 127.0.0.1   | TCP      | 54     | 3304 → 443 [SYN] Seq=0 Win=16384 Len=0                                                                     |

Gambar 2.1. Packet list Mixed1.cap setelah berhasil dimuat pada Wireshark — menampilkan seluruh 19 packet beserta kolom Length dan Protocol.

## 2.4 Penjelasan Setup

| Parameter     | Keterangan                                                                                                                                                                                                                                        |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interface     | Tidak ada capture interface aktif yang digunakan — analisis dilakukan terhadap file PCAP yang sudah ada (offline analysis), bukan live capture                                                                                                    |
| Filter        | Tidak menggunakan display filter khusus; seluruh 19 packet diamati tanpa filter agar variasi ukuran packet dapat diobservasi secara menyeluruh dari awal hingga akhir sesi                                                                        |
| Duration      | Rentang waktu capture pada file: 0.000000 s sampai dengan 64.182000 s (durasi total ± 64,18 detik), mencakup tiga sesi koneksi TCP berbeda                                                                                                        |
| Jumlah Packet | 19 packet, terdiri dari kombinasi protokol TCP (12 packet), DISCARD (1 packet), SSLv2 (1 packet), dan SSLv3 (5 packet)                                                                                                                            |
| Sumber File   | <a href="https://wiki.wireshark.org/SampleCaptures">https://wiki.wireshark.org/SampleCaptures</a> — sample capture resmi dari tim Wireshark, dipilih karena representatif memuat campuran ukuran packet kecil (kontrol) dan besar (handshake TLS) |

## BAB 3: Analisis Skenario & Temuan

### 3.1 Dasar Teori: Struktur Frame, Header, dan Payload

Setiap frame yang melintas di jaringan Ethernet tersusun dari beberapa lapisan header yang saling membungkus (encapsulation), mengikuti model referensi OSI/TCP-IP. Dari luar ke dalam, urutannya adalah: (1) Ethernet II Header (Data Link Layer) — memuat alamat fisik/MAC pengirim dan penerima; (2) IPv4 Header (Network Layer) — memuat alamat logis/IP pengirim dan penerima serta metadata routing; (3) TCP/UDP Header (Transport Layer) — memuat informasi port dan mekanisme reliabilitas transmisi (untuk TCP: sequence number, acknowledgment, flags); dan (4) Payload/Data (Application Layer) — data aktual yang ingin dikirimkan oleh aplikasi, misalnya isi request HTTP, data handshake TLS, atau data biner lainnya.

Nilai pada kolom "Length" di packet list Wireshark merepresentasikan ukuran total frame tersebut di atas kabel (on the wire), yaitu penjumlahan seluruh header dari semua lapisan ditambah payload-nya. Dengan mengetahui ukuran header standar pada tiap lapisan, payload dapat diestimasi melalui pengurangan sederhana:  $\text{Estimated Payload} = \text{Total Length} - \text{Total Header Overhead}$ .

Berdasarkan hasil ekspansi layer pada packet-packet sampel (dijelaskan lebih rinci pada subbab 3.3), struktur header standar pada capture Mixed1.cap ini — dengan asumsi tidak ada opsi tambahan (Options) pada IPv4 maupun TCP — adalah sebagai berikut:

| Layer          | Header Size | Penjelasan                                                                                                                                                                           |
|----------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ethernet II    | 14 bytes    | Terdiri dari MAC address tujuan (6 bytes) + MAC address sumber (6 bytes) + EtherType (2 bytes) yang menandakan protokol lapisan di atasnya (di sini: IPv4, 0x0800)                   |
| IPv4           | 20 bytes    | Header dasar IPv4 tanpa opsi tambahan (Options), mencakup Version, Header Length, Total Length, TTL, Protocol, Checksum, serta alamat IP sumber dan tujuan                           |
| TCP            | 20 bytes    | Header dasar TCP tanpa opsi tambahan, mencakup Source/Destination Port, Sequence Number, Acknowledgment Number, Flags, Window Size, dan Checksum                                     |
| Total Overhead | 54 bytes    | <b>Estimated Payload = Total Length (kolom Length pada packet list) – 54 bytes. Formula ini berlaku untuk seluruh packet berbasis TCP/IPv4 tanpa opsi tambahan pada capture ini.</b> |

### 3.2 Output dari Wireshark

Berdasarkan pengamatan pada kolom Length di packet list (Gambar 2.1), ditemukan variasi ukuran packet mulai dari 54 bytes (packet kontrol tanpa data) hingga 1139 bytes (packet SSL handshake yang membawa sertifikat digital). Sepuluh packet berikut dipilih sebagai sampel representatif dengan variasi ukuran dari yang terkecil hingga terbesar, beserta estimasi payload-nya menggunakan formula pada subbab 3.1:

| No. | Total Length | Protocol | Est. Payload | Keterangan Singkat                                |
|-----|--------------|----------|--------------|---------------------------------------------------|
| 1   | 54           | TCP      | 0            | TCP SYN — pembuka koneksi port 3268→7, tanpa data |
| 6   | 54           | TCP      | 0            | TCP SYN — pembuka koneksi baru port 3280→9        |

| No. | Total Length | Protocol | Est. Payload | Keterangan Singkat                                                                         |
|-----|--------------|----------|--------------|--------------------------------------------------------------------------------------------|
| 19  | 54           | TCP      | 0            | TCP SYN — pembuka koneksi baru port 3304→443 (awal sesi HTTPS)                             |
| 4   | 55           | TCP      | 1            | TCP Keep-Alive dengan 1 byte data pemicu (probe)                                           |
| 10  | 56           | DISCARD  | 2            | Protokol DISCARD, disertai indikasi packet loss (Previous segment not captured)            |
| 18  | 77           | SSLv3    | 23           | Encrypted Alert — notifikasi akhir sesi TLS terenkripsi                                    |
| 17  | 121          | SSLv3    | 67           | Change Cipher Spec + Encrypted Handshake Message                                           |
| 14  | 132          | SSLv2    | 78           | Client Hello — permintaan awal client memulai TLS handshake                                |
| 16  | 258          | SSLv3    | 204          | Client Key Exchange + Change Cipher Spec + Encrypted Handshake                             |
| 15  | 1139         | SSLv3    | 1085         | Server Hello, Certificate, Server Hello Done — packet terbesar, membawa sertifikat digital |

*Catatan penting: untuk packet dengan protokol SSLv2/SSLv3 (No. 14–18), estimasi payload di atas dihitung terhadap batas layer TCP (Total Length – 54). Nilai tersebut sebenarnya berisi SSL/TLS record header (umumnya 5 bytes per record: 1 byte Content Type, 2 byte Version, 2 byte Length) ditambah data handshake aktual (Client Hello, Certificate, dsb.). Dengan kata lain, sebagian kecil dari nilai "Estimated Payload" pada packet-packet tersebut sebenarnya adalah overhead protokol SSL/TLS itu sendiri, bukan murni data aplikasi di atasnya — namun karena SSL/TLS record header berukuran jauh lebih kecil dibanding total datanya, estimasi ini tetap cukup representatif untuk tujuan analisis pada laporan ini.*

### 3.3 Detail Packet Inspection

Untuk memvalidasi formula overhead pada subbab 3.1 secara langsung (bukan sekadar asumsi teoretis), dilakukan ekspansi layer pada dua packet yang mewakili dua ekstrem ukuran dalam sampel: Packet No. 1 (packet terkecil, 54 bytes) dan Packet No. 15 (packet terbesar, 1139 bytes).

#### *a) Packet No. 1 — TCP SYN (54 bytes)*

Screenshot berikut menunjukkan hasil ekspansi layer Frame, Ethernet II, dan Internet Protocol Version 4 pada Packet No. 1. Terlihat pada layer Internet Protocol Version 4: Header Length = 20 bytes, dan Total Length = 40 bytes. Nilai "Total Length" pada layer IPv4 ini merepresentasikan panjang IP datagram saja (IP header 20 bytes + TCP header 20 bytes = 40 bytes), belum termasuk header Ethernet 14 bytes. Dengan demikian, Frame Length total = 40 + 14 = 54 bytes — persis sesuai dengan kolom Length pada packet list, sekaligus membuktikan validitas formula overhead 54 bytes yang digunakan pada subbab 3.1 dan 3.2:

```

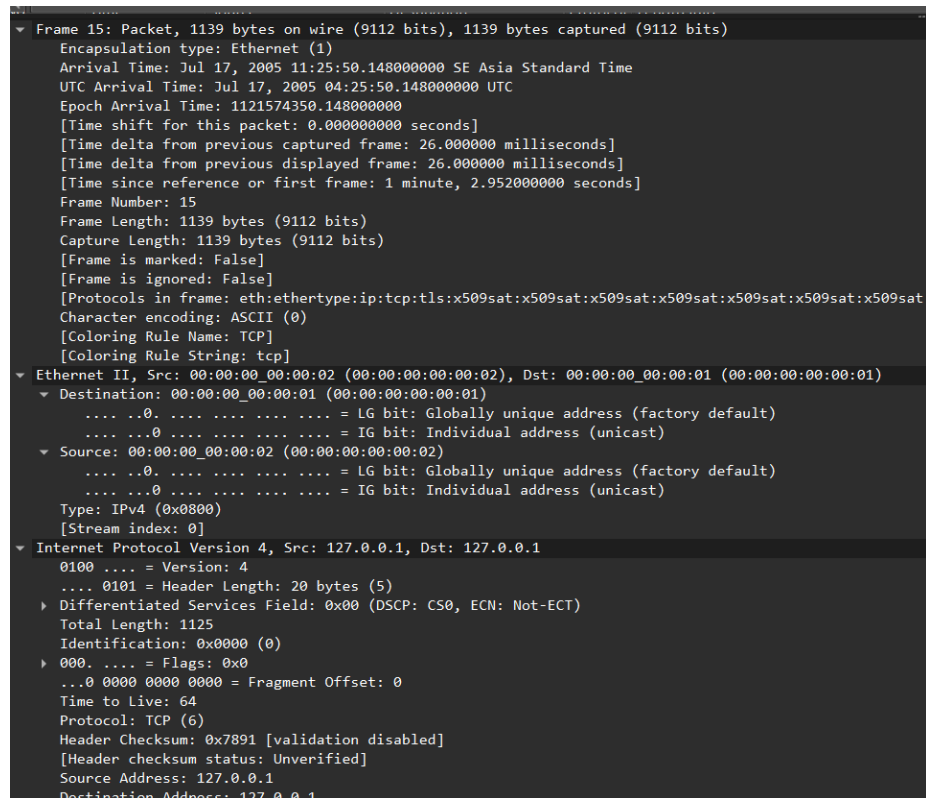
▼ Frame 1: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
  Encapsulation type: Ethernet (1)
  Arrival Time: Jul 17, 2005 11:24:47.196000000 SE Asia Standard Time
  UTC Arrival Time: Jul 17, 2005 04:24:47.196000000 UTC
  Epoch Arrival Time: 1121574287.196000000
  [Time shift for this packet: 0.000000000 seconds]
  [Time since reference or first frame: 0.000000000 seconds]
  Frame Number: 1
  Frame Length: 54 bytes (432 bits)
  Capture Length: 54 bytes (432 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp]
  Character encoding: ASCII (0)
  [Coloring Rule Name: TCP SYN/FIN]
  [Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]
▼ Ethernet II, Src: 00:00:00_00:00:01 (00:00:00:00:00:01), Dst: 00:00:00_00:00:02 (00:00:00:00:00:02)
  ▼ Destination: 00:00:00_00:00:02 (00:00:00:00:00:02)
    .... .. = LG bit: Globally unique address (factory default)
    .... ..0 .... = IG bit: Individual address (unicast)
  ▶ Source: 00:00:00_00:00:01 (00:00:00:00:00:01)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▼ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0x0000 (0)
    ▶ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x7cce [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    [Stream index: 0]
  ▼ Transmission Control Protocol, Src Port: 3268, Dst Port: 7, Seq: 0, Len: 0
    Source Port: 3268
    Destination Port: 7
    Seq: 0
    Len: 0

```

Gambar 3.1. Ekspansi layer Frame, Ethernet II, dan IPv4 pada Packet No. 1 (TCP SYN, 54 bytes). Header Length IPv4 = 20 bytes, Total Length IPv4 = 40 bytes.

*b) Packet No. 15 — SSLv3 Server Hello + Certificate (1139 bytes)*

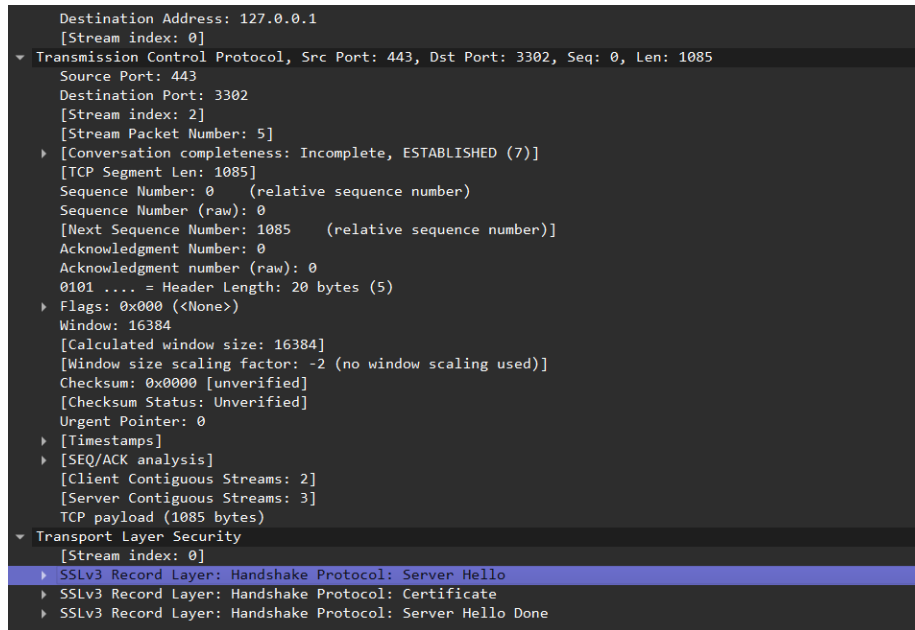
Berikut ekspansi layer pada Packet No. 15 (SSLv3, 1139 bytes) — packet terbesar dalam sampel. Pada layer Internet Protocol Version 4 terlihat Header Length 20 bytes dan Total Length 1125 bytes, konsisten dengan Frame Length 1139 bytes dikurangi 14 bytes header Ethernet ( $1139 - 14 = 1125$ ):



```
▼ Frame 15: Packet, 1139 bytes on wire (9112 bits), 1139 bytes captured (9112 bits)
  Encapsulation type: Ethernet (1)
  Arrival Time: Jul 17, 2005 11:25:50.148000000 SE Asia Standard Time
  UTC Arrival Time: Jul 17, 2005 04:25:50.148000000 UTC
  Epoch Arrival Time: 1121574350.148000000
  [Time shift for this packet: 0.00000000 seconds]
  [Time delta from previous captured frame: 26.000000 milliseconds]
  [Time delta from previous displayed frame: 26.000000 milliseconds]
  [Time since reference or first frame: 1 minute, 2.952000000 seconds]
  Frame Number: 15
  Frame Length: 1139 bytes (9112 bits)
  Capture Length: 1139 bytes (9112 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp:tls:x509sat:x509sat:x509sat:x509sat:x509sat:x509sat:x509sat]
  Character encoding: ASCII (0)
  [Coloring Rule Name: TCP]
  [Coloring Rule String: tcp]
  ▼ Ethernet II, Src: 00:00:00_00:00:02 (00:00:00:00:00:02), Dst: 00:00:00_00:00:01 (00:00:00:00:00:01)
    Destination: 00:00:00_00:00:01 (00:00:00:00:00:01)
      ... ..0. .... = LG bit: Globally unique address (factory default)
      ... ..0. .... = IG bit: Individual address (unicast)
    Source: 00:00:00_00:00:02 (00:00:00:00:00:02)
      ... ..0. .... = LG bit: Globally unique address (factory default)
      ... ..0. .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▼ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 1125
    Identification: 0x0000 (0)
    ▶ 0000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x7891 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
```

*Gambar 3.2. Ekspansi layer Frame, Ethernet II, dan IPv4 pada Packet No. 15. Header Length IPv4 = 20 bytes, Total Length IPv4 = 1125 bytes.*

Lanjutan ekspansi pada layer Transmission Control Protocol menunjukkan baris eksplisit "TCP payload (1085 bytes)" — angka ini sama persis dengan hasil estimasi manual pada tabel subbab 3.2 ( $1139 - 54 = 1085$ ), sekaligus menjadi bukti bahwa Wireshark sendiri mengonfirmasi perhitungan yang dilakukan secara manual pada laporan ini. Di bawahnya, layer Transport Layer Security (SSL/TLS) menunjukkan bahwa 1085 bytes payload tersebut sebenarnya terdiri dari tiga SSLv3 record berbeda yang digabung dalam satu segmen TCP: Handshake Protocol Server Hello, Certificate, dan Server Hello Done. Inilah yang menyebabkan packet ini berukuran jauh lebih besar dibanding packet kontrol TCP biasa — karena membawa data sertifikat digital (public key server beserta certificate chain) yang secara alami berukuran ratusan hingga ribuan byte:



Gambar 3.3. Ekspansi layer TCP (payload 1085 bytes, dikonfirmasi langsung oleh Wireshark) dan Transport Layer Security (SSLv3 Record Layer: Server Hello, Certificate, Server Hello Done) pada Packet No. 15.

### 3.4 Analisis Mendalam per Packet

Berikut penjelasan naratif untuk masing-masing dari 10 packet sampel, disusun agar setiap angka pada tabel 3.2 memiliki konteks dan cerita yang jelas — bagian ini sangat berguna sebagai bahan penjelasan lisan saat sesi presentasi:

- Packet No. 1, 6, dan 19 (masing-masing 54 bytes, Len=0): ketiganya adalah TCP SYN murni yang menjadi paket pembuka dari tiga sesi koneksi TCP berbeda dalam capture ini (port 3268→7, 3280→9, dan 3304→443). Karena SYN hanya berfungsi sebagai sinyal "saya ingin membuka koneksi" tanpa membawa data aplikasi apa pun, ukurannya selalu identik dengan total header murni (54 bytes) — inilah packet dengan ukuran paling minimum yang mungkin muncul pada koneksi TCP/IPv4 standar.
- Packet No. 4 (55 bytes, Len=1): merupakan packet TCP Keep-Alive dengan 1 byte data "dummy". Keep-Alive digunakan untuk menjaga koneksi tetap terbuka (mencegah timeout) di tengah periode idle; penambahan 1 byte data (biasanya byte kosong/0x00) adalah teknik standar agar packet ini direspons dengan ACK oleh pihak lawan, sehingga pengirim dapat memverifikasi koneksi masih hidup.
- Packet No. 10 (56 bytes): tercatat dengan protokol DISCARD dan disertai flag analisis Wireshark "[TCP Previous segment not captured]". Protokol DISCARD sendiri (RFC 863) adalah layanan sederhana yang membuang setiap data yang diterimanya — namun yang lebih penting untuk dicermati adalah tanda "Previous segment not captured", yang berarti Wireshark mendeteksi ada gap pada sequence number TCP, mengindikasikan bahwa proses capture tidak berhasil menangkap satu atau lebih segmen sebelumnya secara utuh.
- Packet No. 14 (132 bytes): SSLv2 Client Hello — pesan pertama dalam proses TLS handshake, dikirim oleh client untuk memberi tahu server mengenai versi SSL/TLS,

cipher suite, dan session ID yang didukung/diinginkan client. Ukurannya lebih besar dari packet TCP kontrol biasa karena memuat daftar cipher suite yang cukup panjang.

- Packet No. 15 (1139 bytes) — dibahas detail pada subbab 3.3 poin (b): packet terbesar dalam keseluruhan sampel, berisi respons server atas Client Hello berupa Server Hello (pemilihan cipher suite final), Certificate (rantai sertifikat digital X.509 milik server), dan Server Hello Done (penanda akhir giliran server pada tahap ini).
- Packet No. 16 (258 bytes): melanjutkan handshake dengan Client Key Exchange (client mengirimkan kunci pra-master yang dienkripsi menggunakan public key server dari sertifikat sebelumnya), Change Cipher Spec (penanda bahwa mulai saat ini komunikasi akan dienkripsi), dan awal dari Encrypted Handshake Message.
- Packet No. 17 (121 bytes): lanjutan Change Cipher Spec dan Encrypted Handshake Message dari sisi server, menandakan kedua pihak sudah menyelesaikan pertukaran kunci dan siap berkomunikasi secara terenkripsi penuh.
- Packet No. 18 (77 bytes): sebuah Encrypted Alert — pesan notifikasi (misalnya penutupan sesi atau peringatan tertentu) yang sudah dienkripsi menggunakan kunci sesi yang telah disepakati pada tahap handshake sebelumnya.

Catatan tambahan: packet No. 16, 17, dan 18 sama-sama disertai flag "[TCP Previous segment not captured]", konsisten dengan temuan gap sequence number yang juga ditemukan pada packet No. 10 — mengindikasikan bahwa proses capture pada sesi TLS (port 443) kemungkinan besar tidak menangkap seluruh traffic secara 100% lengkap sejak awal koneksi.

### 3.5 Analisis Mendalam: Temuan, Interpretasi, dan Implikasi

#### *Apa yang ditemukan?*

- Terdapat pola jelas antara jenis/fase komunikasi dengan ukuran packet: packet kontrol murni (SYN, ACK, Keep-Alive) selalu berukuran minimum (54–55 bytes), sedangkan packet yang membawa data handshake TLS atau sertifikat digital berukuran jauh lebih besar (77–1139 bytes).
- Ditemukan indikasi packet loss / capture yang tidak lengkap pada beberapa titik (packet No. 10, 16, 17, 18), ditandai pesan analisis otomatis Wireshark "Previous segment not captured".
- Ditemukan penggunaan protokol kriptografi versi lama, yaitu SSLv2 (pada Client Hello) dan SSLv3 (pada seluruh proses handshake dan komunikasi terenkripsi selanjutnya).

#### *Bagaimana interpretasinya?*

Pola ukuran packet yang ditemukan berkorelasi langsung dengan fase komunikasi yang sedang berlangsung: fase pembukaan koneksi (three-way handshake TCP) secara inheren tidak memerlukan data apa pun sehingga selalu minimum size, sedangkan fase pertukaran kunci kriptografi dan sertifikat pada TLS handshake secara alami menghasilkan packet besar karena harus membawa data kriptografi (public key, certificate chain, encrypted pre-master secret) yang ukurannya memang signifikan menurut standar kriptografi modern. Munculnya pesan "Previous segment not captured" secara berulang mengindikasikan bahwa proses perekaman traffic ini kemungkinan besar dimulai setelah sebagian traffic sudah berjalan (capture dimulai terlambat),



atau terjadi packet drop pada titik pengambilan data (misalnya karena buffer capture penuh, atau posisi tap/mirror port yang tidak menangkap seluruh arah traffic).

#### *Apa implikasi network/security-nya?*

- Implikasi keamanan positif: terlihat transisi yang jelas dari komunikasi plaintext (TCP handshake awal pada port 443) menuju komunikasi terenkripsi penuh (SSLv3 Change Cipher Spec dan seterusnya) — ini menunjukkan penerapan enkripsi transport layer untuk melindungi kerahasiaan data yang dikirim antara client dan server, sesuai prinsip dasar keamanan jaringan (confidentiality).
- Implikasi keamanan negatif (risiko): penggunaan SSLv3 pada capture ini adalah catatan keamanan serius apabila ditemukan pada sistem produksi masa kini. SSLv3 telah dinyatakan usang oleh IETF (RFC 7568, tahun 2015) dan memiliki kerentanan terdokumentasi bernama POODLE (Padding Oracle On Downgraded Legacy Encryption) yang memungkinkan penyerang membaca sebagian data terenkripsi melalui manipulasi padding cipher block. Sistem modern seharusnya menggunakan minimal TLS 1.2, idealnya TLS 1.3.
- Implikasi terhadap integritas forensik: packet loss yang terindikasi (DISCARD, "Previous segment not captured") dapat menyulitkan proses investigasi forensik jaringan apabila capture semacam ini digunakan untuk menyelidiki insiden keamanan nyata, karena sebagian konteks komunikasi (isi segmen yang hilang) tidak dapat direkonstruksi ulang dari file capture yang ada.
- Implikasi terhadap performa jaringan: campuran packet kecil (kontrol, rasio overhead terhadap payload sangat tinggi/100%) dan packet besar (data, rasio overhead terhadap payload sangat rendah/±4,7% untuk packet No. 15) menunjukkan pola trafik yang wajar terjadi pada koneksi TCP/TLS pada umumnya. Namun demikian, pada skala jaringan yang jauh lebih besar (ribuan hingga jutaan koneksi), proporsi overhead pada packet-packet kecil tetap perlu diperhitungkan secara serius dalam analisis efisiensi bandwidth dan kapasitas link jaringan.

### 3.6 Ringkasan Temuan (Tabel)

| Kategori             | Temuan                                                                                                                                | Contoh Packet                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Packet terkecil      | TCP SYN tanpa data (Len=0), 100% overhead header, 0% payload                                                                          | No. 1, 6, 19 (54 bytes)               |
| Packet terbesar      | SSL Server Hello + Certificate, ±95,3% payload (1085/1139 bytes)                                                                      | No. 15 (1139 bytes)                   |
| Anomali              | Protokol DISCARD & pesan packet loss "Previous segment not captured" berulang                                                         | No. 10, 16, 17, 18                    |
| Risiko keamanan      | Penggunaan protokol SSLv2/SSLv3 yang sudah usang & rentan (mis. POODLE)                                                               | No. 14–18                             |
| Validasi perhitungan | Estimasi payload manual (Total Length – 54) terkonfirmasi identik dengan nilai "TCP payload" yang ditampilkan langsung oleh Wireshark | No. 1 & No. 15 (lihat Gambar 3.1–3.3) |

## BAB 4: Cek List Verifikasi Hasil Instalasi

Bab ini menyajikan lima item verifikasi wajib yang membuktikan bahwa instalasi Wireshark pada WSL2/Linux benar-benar telah dilakukan dan berfungsi sebagaimana mestinya. Setiap item disertai command yang dijalankan, penjelasan mengapa item tersebut relevan diverifikasi, status hasil, dan bukti screenshot asli.

| Item Verifikasi        | Command / Evidence                   | Status                                                |
|------------------------|--------------------------------------|-------------------------------------------------------|
| Wireshark version      | wireshark --version                  | Sukses — versi 4.6.4                                  |
| WSL2/Linux OS          | whoami && hostname                   | Sukses — andreas@LAPTOP-IUN0JF8P                      |
| Network interface      | ip a (WSL2)                          | Sukses — interface lo & eth0 terdeteksi               |
| PCAP file location     | ls -lah Mixed1.cap                   | Sukses —<br>/mnt/c/Users/ANDREAS/Downloads/Mixed1.cap |
| Timestamp verification | ls -lah Mixed1.cap / stat Mixed1.cap | Sukses — Jul 16, 11:55                                |

### Evidence Screenshot & Penjelasan Tiap Item

#### 1. Wireshark version

Verifikasi ini penting untuk memastikan versi Wireshark yang terpasang sudah merupakan rilis stabil dan kompatibel dengan seluruh fitur analisis yang dibutuhkan pada praktikum (dissector protokol SSL/TLS, statistik, dsb). Command wireshark --version juga sekaligus menampilkan informasi runtime OS, yang pada kasus ini membuktikan Wireshark berjalan di atas kernel WSL2, bukan Windows native.

```
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ wireshark --version
Wireshark 4.6.4.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
  GLib: 2.87.3
  With:
    +brotli                               +nghttp2 1.68.0
    +Gcrypt 1.12.0                       +nghttp3 1.12.0
    +GnuTLS 3.8.12 and PKCS#11           +PCRE2 10.46 2025-08-27
    +Kerberos (MIT)                     +POSIX capabilities (Linux)
    +libnl 3                             +Qt 6.10.2
    +libpcap                             +QtDBus
    +libsmi 0.4.8                       +QtMultimedia
    +libxml2 2.15.1                     +Snappy 1.2.2
    +Lua 5.4.8                          +xxhash 0.8.3
    +LZ4 1.10.0                         +zlib 1.3.1
    +MaxMind                            +Zstandard 1.5.7
  Without:
    -automatic updates -zlib-ng

Runtime info:
  OS: Linux 6.18.33.2-microsoft-standard-WSL2
```

Output 'wireshark --version' — Wireshark 4.6.4 pada WSL2, dikonfirmasi melalui baris runtime OS: Linux ...-microsoft-standard-WSL2.

#### 2. WSL2/Linux OS

Verifikasi ini membuktikan bahwa seluruh proses instalasi dan pekerjaan command-line memang dilakukan di dalam lingkungan Linux (WSL2), bukan di PowerShell/CMD Windows. Command whoami menampilkan user Linux yang sedang aktif, sedangkan hostname menampilkan nama mesin yang terdaftar pada sesi WSL2 tersebut.

```
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ whoami && hostname
andreas
LAPTOP-IUN0JF8P
```

Output 'whoami && hostname' — user 'andreas' pada hostname 'LAPTOP-IUN0JF8P', menandakan sesi shell benar berjalan di dalam WSL2 Ubuntu.

### 3. Network interface

Verifikasi ini membuktikan bahwa lingkungan WSL2 memiliki stack jaringan (networking stack) yang aktif dan berfungsi, sebagai prasyarat teoretis untuk dapat melakukan packet capture (meskipun pada praktikum ini capture dilakukan secara offline terhadap file PCAP, bukan live capture — lihat BAB 1.5). Command `ip a` menampilkan seluruh interface jaringan yang dikenali kernel Linux pada WSL2.

```
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1492 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:98:8f:0a brd ff:ff:ff:ff:ff:ff
    altname enx00155d988f0a
    inet 172.26.118.254/20 brd 172.26.127.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe98:8f0a/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ |
```

Output 'ip a' pada WSL2 — interface `lo` (loopback, 127.0.0.1) dan `eth0` (NAT ke jaringan Windows host, IP 172.26.118.254) terdeteksi aktif.

### 4. PCAP file location

Verifikasi ini membuktikan bahwa file sample capture `Mixed1.cap` yang dianalisis pada laporan ini benar-benar ada dan dapat diakses dari sistem file, sekaligus menunjukkan lokasi penyimpanannya. Command `ls -lah` menampilkan detail file secara human-readable (ukuran dalam KB/MB, bukan hanya byte mentah).

```
PS C:\Users\ANDREAS> wsl
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ cd /mnt/c/Users/ANDREAS/Downloads
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS/Downloads$ ls -lah Mixed1.cap
-rwxrwxrwx 1 andreas andreas 15K Jul 16 11:55 Mixed1.cap
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS/Downloads$ |
```

Output 'ls -lah Mixed1.cap' — file ditemukan di `/mnt/c/Users/ANDREAS/Downloads` (diakses dari WSL2 melalui mount point Windows), berukuran 15K, dengan permission `-rwxrwxrwx`.

### 5. Timestamp verification

Verifikasi ini memastikan bahwa file yang dianalisis merupakan unduhan yang relatif baru/wajar (bukan file lama yang mungkin sudah tidak representatif atau file yang dimanipulasi), dengan melihat kolom timestamp modifikasi terakhir pada output `ls -lah` yang sama dengan item nomor 4.

```
PS C:\Users\ANDREAS> wsl
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS$ cd /mnt/c/Users/ANDREAS/Downloads
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS/Downloads$ ls -lah Mixed1.cap
-rwxrwxrwx 1 andreas andreas 15K Jul 16 11:55 Mixed1.cap
andreas@LAPTOP-IUN0JF8P:/mnt/c/Users/ANDREAS/Downloads$ |
```

*Kolom timestamp pada output 'ls -lah' menunjukkan file Mixed1.cap terakhir dimodifikasi pada 16 Juli, pukul 11:55 — menandakan file merupakan hasil unduhan yang baru dan konsisten dengan waktu pengerjaan praktikum ini.*

## BAB 5: Kesimpulan & Rekomendasi

### 5.1 Ringkasan Temuan Utama

- Ukuran packet pada capture Mixed1.cap bervariasi cukup ekstrem, mulai dari 54 bytes (packet kontrol TCP tanpa payload) hingga 1139 bytes (packet SSL handshake berisi sertifikat digital) — rentang variasi lebih dari 21 kali lipat.
- Header overhead standar pada capture ini terkonfirmasi sebesar 54 bytes (Ethernet 14 + IPv4 20 + TCP 20 bytes), dibuktikan baik melalui perhitungan manual maupun melalui nilai "TCP payload" yang ditampilkan langsung oleh Wireshark pada Packet No. 15 (lihat Gambar 3.3).
- Ditemukan pola yang sangat jelas dan konsisten: packet kontrol (SYN, ACK, Keep-Alive) selalu berukuran minimum, sedangkan packet yang membawa data aplikasi atau data kriptografi (sertifikat TLS) berukuran signifikan lebih besar, sejalan dengan teori encapsulation protokol jaringan pada subbab 3.1.

### 5.2 Anomali / Insight yang Terdeteksi

Ya, terdapat beberapa anomali yang terdeteksi selama analisis, sebagai berikut:

- Protokol DISCARD pada packet No. 10 beserta pesan "Previous segment not captured" yang berulang pada packet No. 16, 17, dan 18 — mengindikasikan adanya packet loss atau proses capture yang tidak menangkap seluruh segmen komunikasi secara utuh sejak awal sesi.
- Penggunaan protokol SSLv2 (pada Client Hello, packet No. 14) dan SSLv3 (packet No. 15–18) — kedua protokol ini sudah dianggap usang secara resmi oleh standar industri (RFC 6176 mendepresiasi SSLv2, RFC 7568 mendepresiasi SSLv3) dan memiliki kerentanan keamanan yang diketahui publik dan terdokumentasi.

### 5.3 Implikasi Keamanan

- Penggunaan SSLv2/SSLv3 berisiko terhadap serangan downgrade dan kerentanan terdokumentasi seperti POODLE (pada SSLv3) maupun kelemahan struktural bawaan SSLv2 (seperti tidak adanya proteksi integritas pada handshake), sehingga sistem yang masih menggunakan protokol ini pada lingkungan produksi perlu segera dimigrasikan ke TLS 1.2 atau idealnya TLS 1.3.
- Packet loss yang terdeteksi dapat menyulitkan proses investigasi forensik jaringan apabila capture semacam ini digunakan untuk menyelidiki insiden keamanan nyata, karena sebagian konteks komunikasi (segmen yang hilang) tidak dapat direkonstruksi ulang dari data yang tersedia.
- Variasi ukuran packet yang signifikan (54 hingga 1139 bytes) dalam satu sesi komunikasi pada dasarnya adalah hal yang wajar secara teknis, namun tetap perlu dipantau secara berkelanjutan pada sistem produksi nyata, karena pola ukuran packet yang tidak lazim (misalnya packet yang jauh melebihi ukuran biasa secara konsisten) juga dapat menjadi salah satu indikator aktivitas anomali seperti data exfiltration atau tunneling terselubung.

## 5.4 Keterbatasan Analisis

- Analisis pada laporan ini bersifat offline (menggunakan file PCAP sampel yang sudah ada), bukan live capture, sehingga tidak mencerminkan kondisi traffic real-time pada jaringan produksi yang sesungguhnya sedang berjalan.
- Sample Mixed1.cap yang digunakan berasal dari sumber publik untuk keperluan edukasi, dengan volume packet yang relatif kecil (19 packet) dan durasi singkat ( $\pm 64$  detik) — sehingga generalisasi temuan terhadap pola trafik jaringan skala besar/enterprise perlu dilakukan dengan hati-hati.
- Estimasi payload SSL/TLS pada laporan ini dihitung terhadap batas layer TCP, tanpa memisahkan secara presisi overhead SSL/TLS record header ( $\pm 5$  bytes per record) dari data handshake aktual, sehingga angka "Estimated Payload" pada packet SSL/TLS bersifat mendekati (approximate), bukan nilai eksak murni application-layer data.

## 5.5 Lessons Learned & Takeaways

- Kolom Length pada Wireshark adalah entry point yang sangat efektif untuk mengidentifikasi jenis komunikasi (kontrol vs data) sebelum melakukan deep-dive ke detail packet satu per satu — sebuah teknik triase awal yang efisien dalam analisis traffic jaringan skala besar.
- Memahami struktur header tiap layer (Ethernet, IP, TCP, dan opsional TLS) adalah keterampilan fundamental yang memungkinkan estimasi payload secara akurat dan membedakan overhead protokol murni dari data aplikasi yang sesungguhnya dikirim — keterampilan ini secara langsung dapat diverifikasi silang dengan nilai yang ditampilkan Wireshark sendiri (seperti "TCP payload"), sebagaimana didemonstrasikan pada subbab 3.3.
- Kombinasi analisis ukuran packet dengan protokol dan flag tambahan (SYN, ACK, [Previous segment not captured]) memberikan gambaran investigasi yang jauh lebih lengkap dibandingkan hanya melihat satu parameter (misalnya ukuran) secara terisolasi.
- Ditemukannya protokol SSL versi lama pada capture ini menjadi pengingat penting akan perlunya audit protokol keamanan secara berkala pada infrastruktur jaringan nyata, karena protokol yang sudah usang dapat tetap 'bersembunyi' pada sistem legacy tanpa disadari hingga suatu saat dieksploitasi.
- Proses instalasi Wireshark pada WSL2 (BAB 1) sekaligus melatih kemampuan troubleshooting nyata di lingkungan Linux (permission, group membership, virtualisasi) — sebuah pengalaman praktis yang relevan dengan pekerjaan sehari-hari seorang praktisi keamanan siber maupun network administrator.

## GLOSARIUM ISTILAH

Bagian ini disediakan sebagai referensi cepat istilah-istilah teknis yang digunakan sepanjang laporan, untuk memudahkan pemahaman saat sesi presentasi maupun bagi pembaca yang kurang familiar dengan istilah jaringan.

| Istilah                              | Penjelasan                                                                                                                                                                                      |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WSL2</b>                          | Windows Subsystem for Linux versi 2 — fitur Windows yang menjalankan kernel Linux sungguhan di atas Windows tanpa mesin virtual manual terpisah.                                                |
| <b>PCAP</b>                          | Packet Capture — format file standar untuk menyimpan rekaman traffic jaringan lengkap dengan timestamp dan data mentah tiap frame.                                                              |
| <b>Frame</b>                         | Satu unit data lengkap yang melintas di jaringan Ethernet, mencakup seluruh header (Ethernet, IP, TCP/UDP) ditambah payload/data aplikasi.                                                      |
| <b>Header Overhead</b>               | Jumlah byte yang digunakan oleh header protokol (bukan data aplikasi) pada suatu frame/packet.                                                                                                  |
| <b>Payload</b>                       | Data aktual yang dikirim oleh aplikasi, terletak setelah seluruh header protokol pada suatu packet.                                                                                             |
| <b>TCP SYN</b>                       | Packet pembuka pada proses TCP three-way handshake, menandakan permintaan pembukaan koneksi baru.                                                                                               |
| <b>Keep-Alive</b>                    | Mekanisme TCP untuk menjaga koneksi tetap terbuka selama periode idle dengan mengirim packet probe kecil secara berkala.                                                                        |
| <b>SSL/TLS Handshake</b>             | Rangkaian pertukaran pesan antara client dan server untuk menyepakati parameter enkripsi (cipher suite, kunci sesi) sebelum komunikasi data terenkripsi dimulai.                                |
| <b>Certificate (X.509)</b>           | Sertifikat digital yang memuat public key server beserta identitasnya, digunakan client untuk memverifikasi keaslian server dalam proses TLS handshake.                                         |
| <b>POODLE</b>                        | Padding Oracle On Downgraded Legacy Encryption — kerentanan keamanan pada protokol SSLv3 yang memungkinkan penyerang membaca sebagian data terenkripsi melalui manipulasi padding cipher block. |
| <b>Previous segment not captured</b> | Pesan analisis otomatis Wireshark yang menandakan ditemukannya gap pada sequence number TCP, mengindikasikan ada segmen yang tidak berhasil direkam saat proses capture.                        |
| <b>MTU</b>                           | Maximum Transmission Unit — ukuran maksimum suatu frame yang dapat dikirim melalui suatu media jaringan (umumnya 1500 bytes untuk Ethernet standar).                                            |